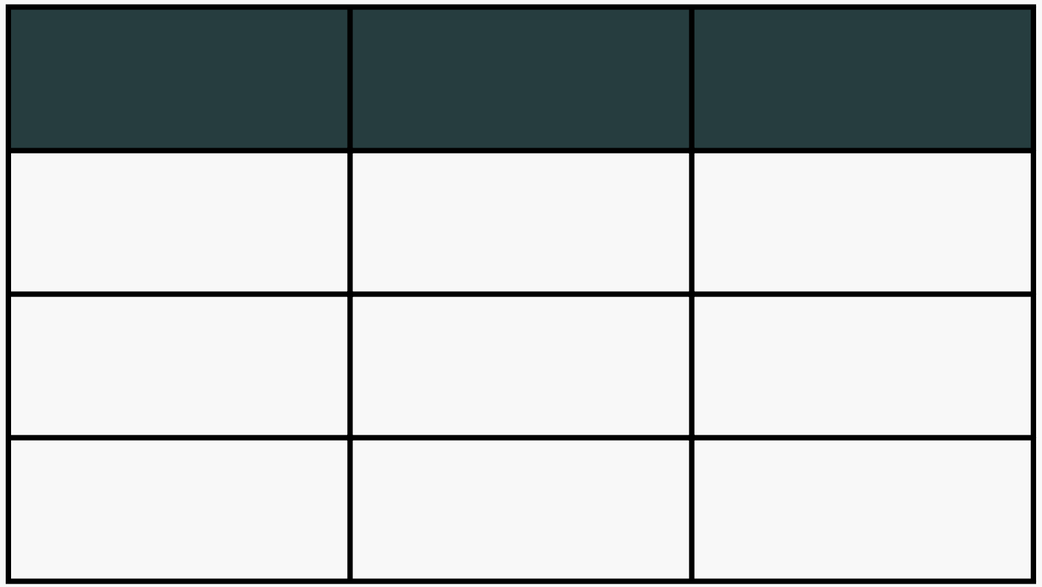
DATA & STORAGE

TYPES OF DATABASES

- **Relational (SQL)**
- **Non-Relational (NoSQL)**
- Objet Oriented
- Distributed
- Data Warehouse
- Graph
- Multimodel
- Document/JSON
- Self-driving

RELATIONAL (SQL) DBS

- Data is stored in tables that have rows and columns
- Each table represents an entity



RELATIONAL (SQL) DBS

Schemas - used to describe how the tables within a db are related

- Relationship types --> 1:1, many:1, many:many

Primary Key - identifier for a record within the table

Foreign Key - identifier that is a primary key in another table (for relating records of one table with records from another)

WHAT IS SQL?

- Structured Querying Language
    - Language that is based on relational algebra
- Syntax varies depending on RDBMS
    - MySQL
    - PostgreSQL
    - Oracle
    - Microsoft SQL Server

SQL STATEMENTS & CRUD

- Create --> INSERT
- Read --> SELECT
- Update --> UPDATE
- Delete --> DELETE

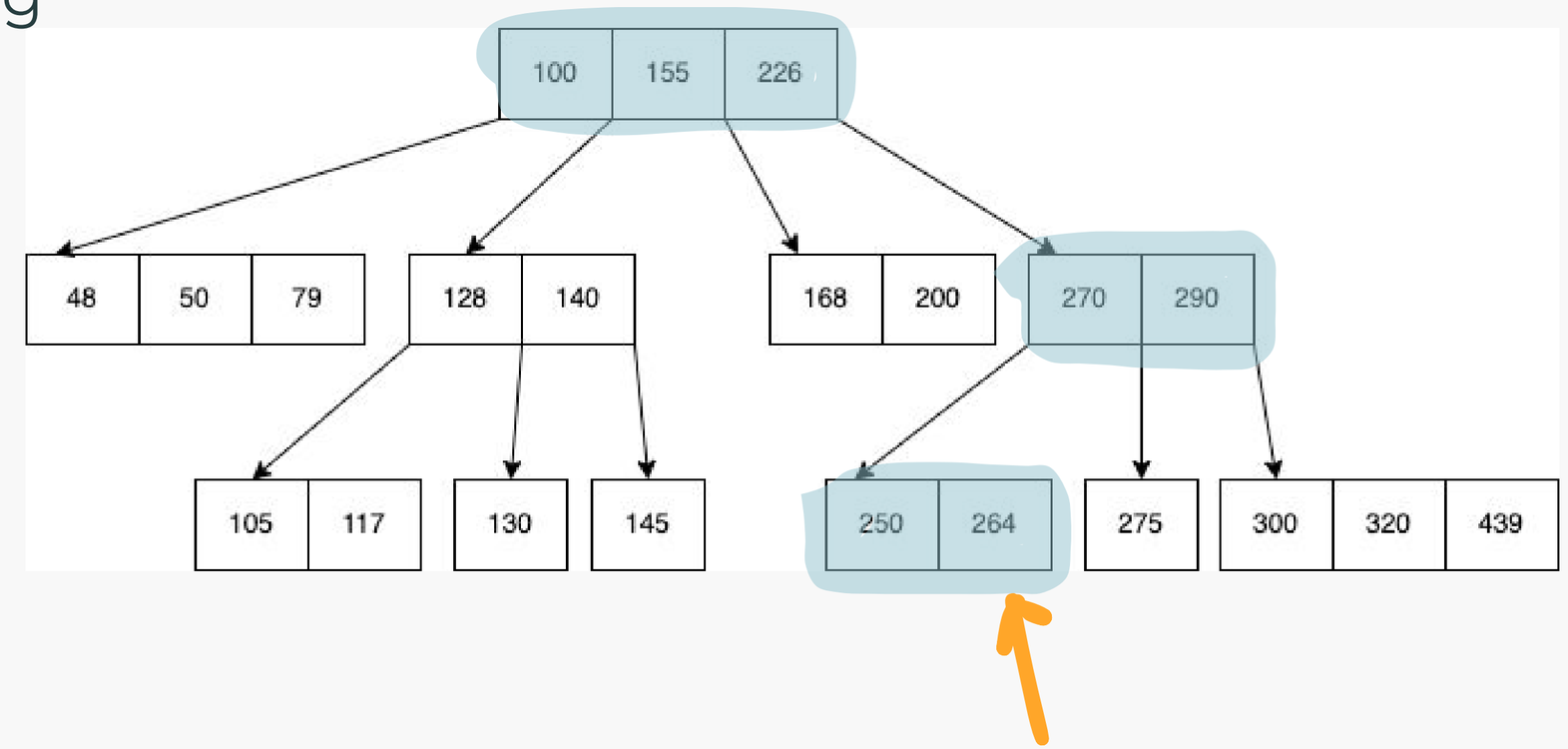
JOINS

Allows us to query information from multiple tables

- Inner
- Left
- Right
- Full/Outer

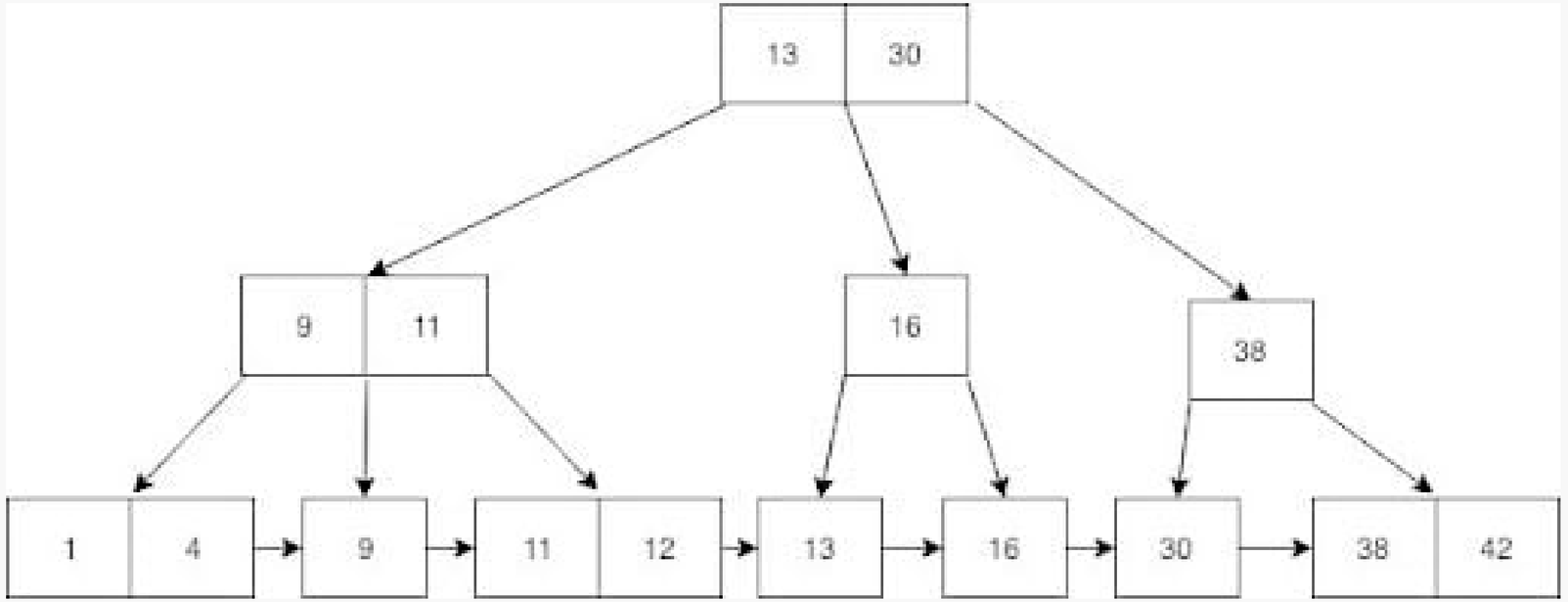
B TREES

- SQL dbs use tree indexing
- Balanced search trees
- Nodes can have numerous children
- Meant to handle large volume of data



B+ TREES

- All non-leaf node values are duplicated in leaf nodes



ACID COMPLIANCE

- Atomicity
- Consistency
- Isolation
- Durability

When transaction happens, data validity is guaranteed

ACID COMPLIANCE

Atomicity - the all or nothing approach

- Db transactions usually happen in multiple steps
- Either all of the steps succeed or none of them do

Ex: Final Project payment terms - as the money is leaving the group member’s account to the group leader

ACID COMPLIANCE

Consistency

- Data is valid before and after the transaction (doesn’t go against any constraints)

Ex: Final project payment - if you don’t have any more money in your account then the transaction fails

ACID COMPLIANCE

Isolation

- All transactions run in an isolated environment and don’t interfere with one another

Ex: If you have $100 in your payment account to use, and you’re paying for 2 groups at the same time ($50/each) then your net is $0 once the transaction is completed.

ACID COMPLIANCE

Durability

- When a transaction is completed, the changes in transaction are in the db
- This should still be the case regardless of DR (ex: power failures, etc)

DB NORMALIZATION

- Goal: Table can’t express redundant information so only has one version of the truth
    - Reduces inefficiences

NON-RELATIONAL (NOSQL) DBS

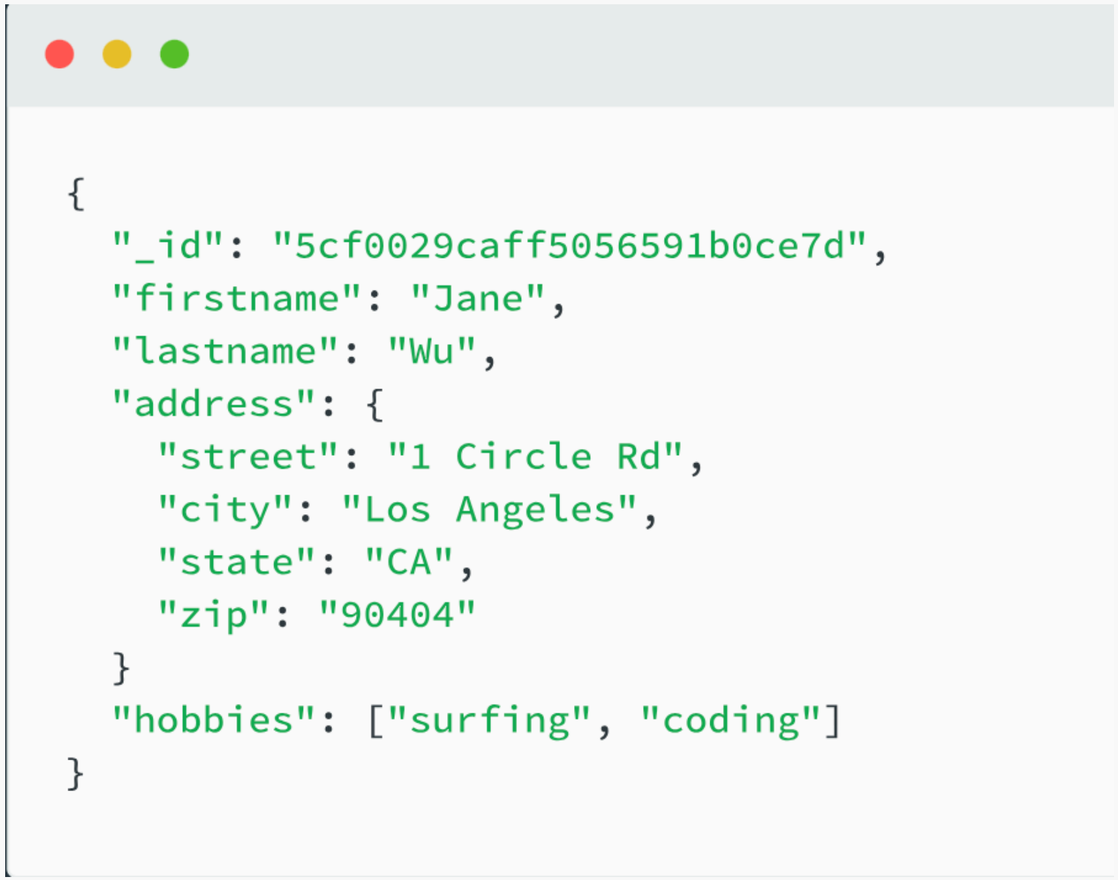
- Data isn’t stored in tables
- Data isn’t strictly represented with relationships
  - Document
  - Graph
  - Key-Value

NON-RELATIONAL (NOSQL) DBS

Document DB - stores info as documents that contain field-value pairs

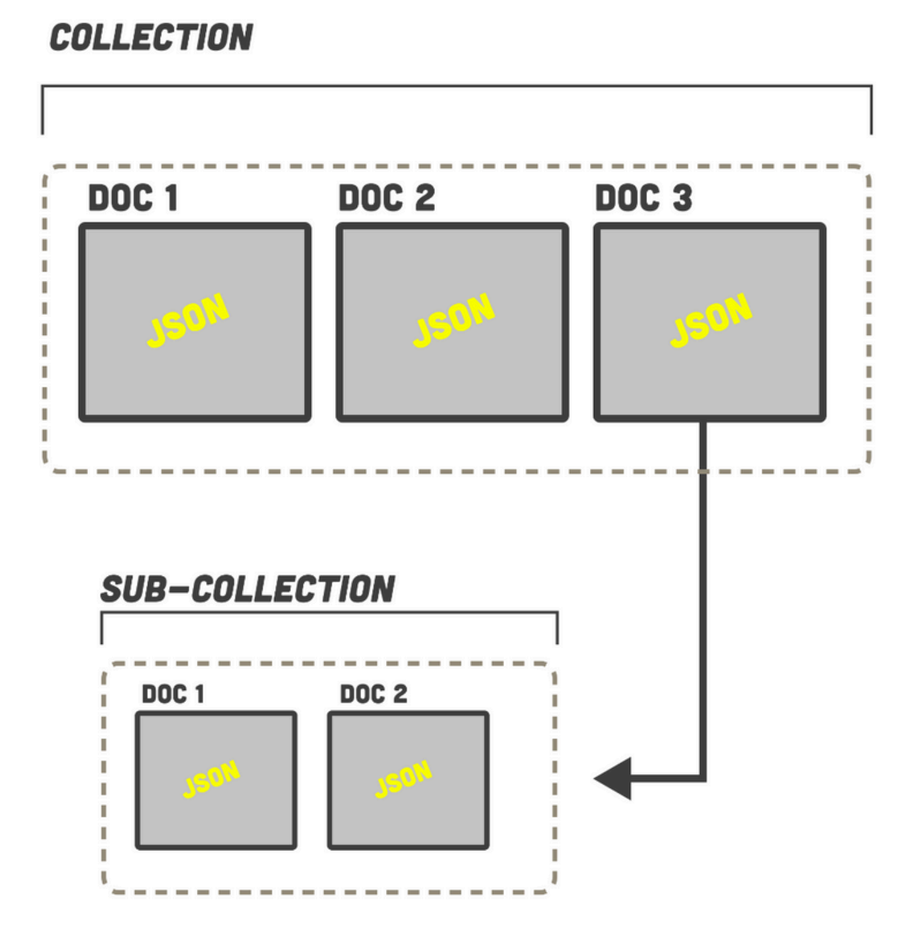
Examples:

- Elasticsearch
- MongoDB



```
{
  "_id": "5cf0029caff5056591b0ce7d",
  "firstname": "Jane",
  "lastname": "Wu",
  "address": {
    "street": "1 Circle Rd",
    "city": "Los Angeles",
    "state": "CA",
    "zip": "90404"
  }
  "hobbies": ["surfing", "coding"]
}
```

NON-RELATIONAL (NOSQL) DBS



Reads can be faster

Writing/updating data is more complex

- Not great for graphs or networks

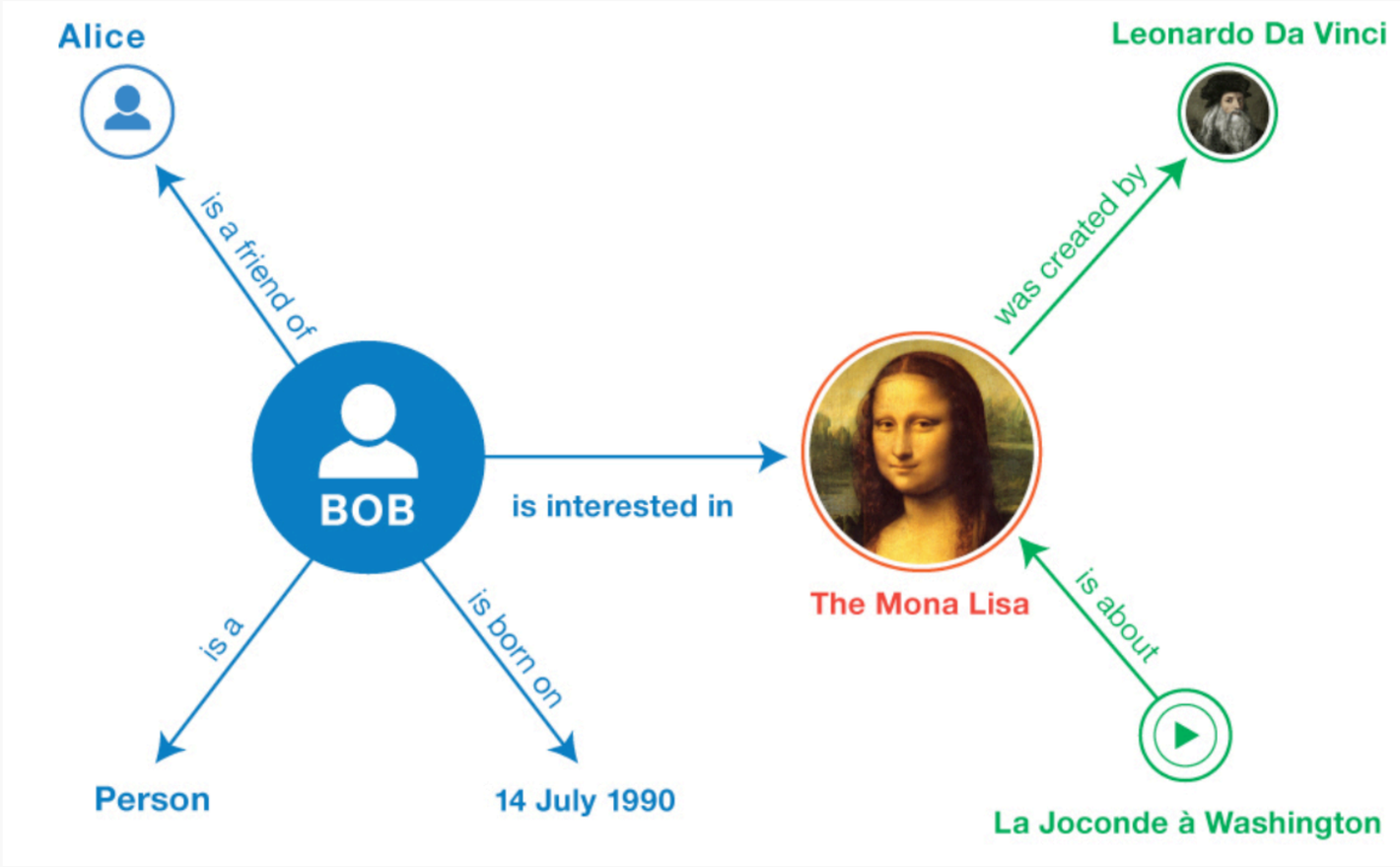
Best for: Games, IoT

NON-RELATIONAL (NOSQL) DBS

Graph DB - storing data as nodes and relationships between those nodes

Example:

- Neo4j
- Amazon Neptune



NON-RELATIONAL (NOSQL) DBS

Key-Value DB - data is stored as a unique key and associated value pair

Examples: Redis

Key	Value
K1	AAA,BBB,CCC
K2	AAA,BBB
K3	AAA,DDD
K4	AAA,2,01/01/2015
K5	3,ZZZ,5623

NON-RELATIONAL (NOSQL) DBS

redis

Key-Value DB - data is held in memory as opposed to disk

```
res = r.set("bike:1", "Process 134")
print(res)
# >>> True

res = r.get("bike:1")
print(res)
# >>> "Process 134"
```

Ideal for:

- Fast reads
  - Ex: Caching
- No querying, no joins

AT PLANET SCALE...

- How do we handle petabytes of data?

DATA STREAMING

When data is emitted continuously for processing (ex: sensor/location data)

- Meant to have low latency data processing

Recall event streaming - (producers/consumers)

- Apache Kafka

DATA LAKES

- The repository for ingested data (ex: streamed data)
- Data is in raw format (object blobs/files)
- Stored in file systems like Hadoop

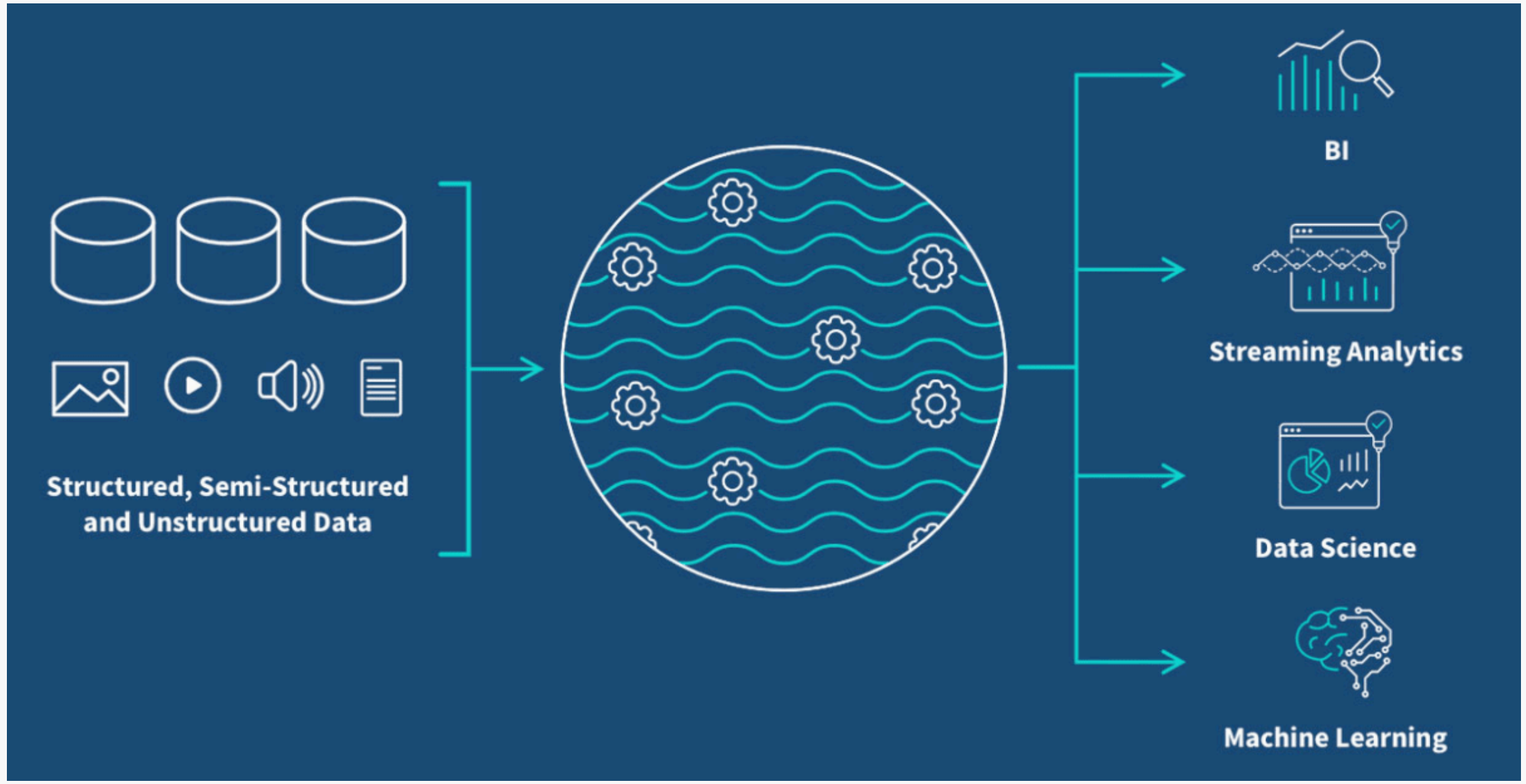


Image source: https://www.qlik.com/us/data-lake

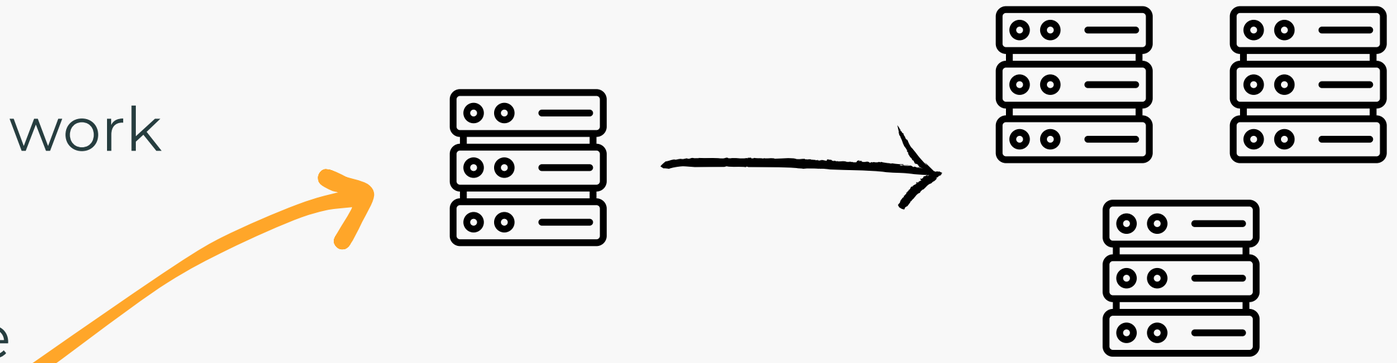
DATA CENTERS

- The physical location that holds the data

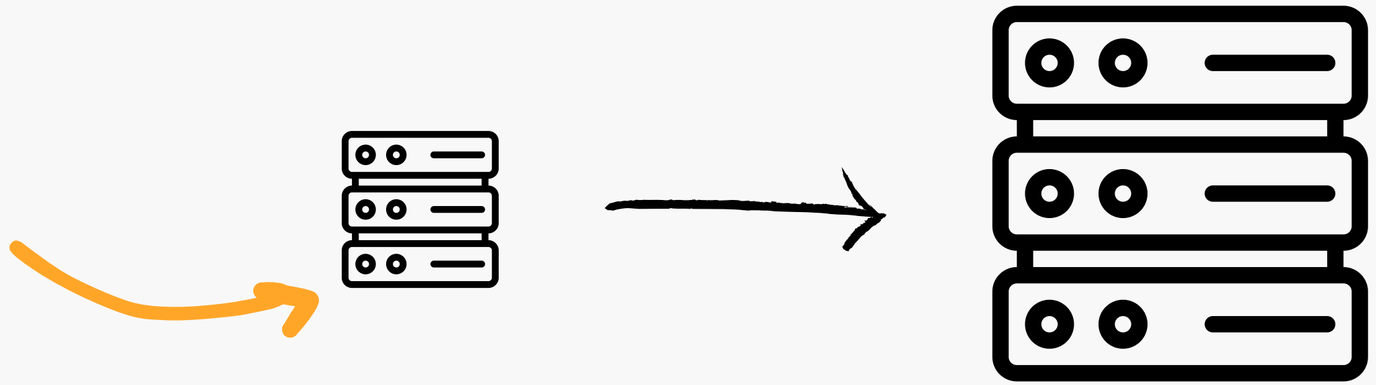
HORIZONTAL VS VERTICAL SCALING

Scaling - to increase the system's ability to handle work

Horizontal - adding more machines to the system



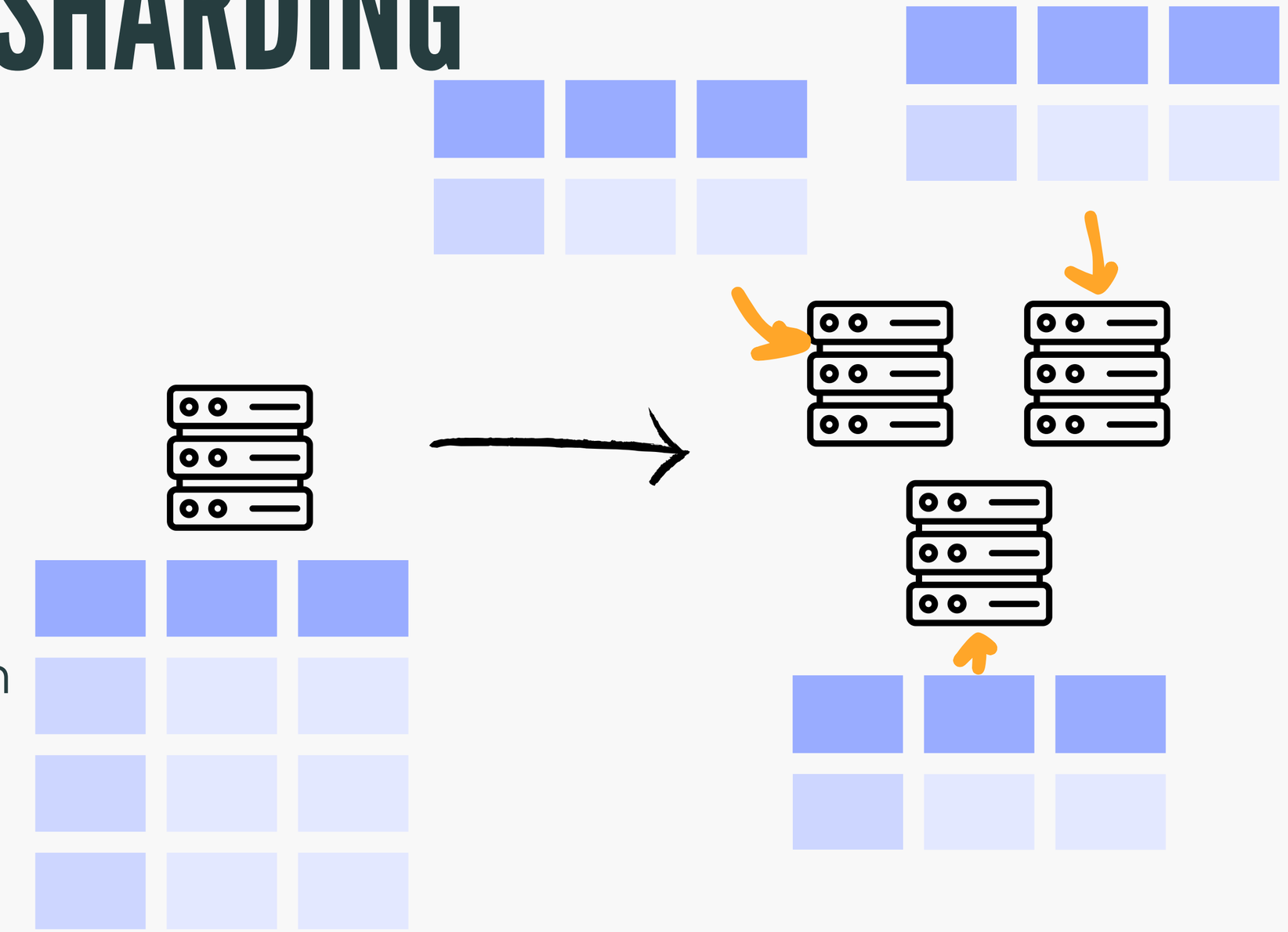
Vertical - increasing the compute power (CPU, RAM, storage, etc) of a machine



PARTIONING/SHARDING

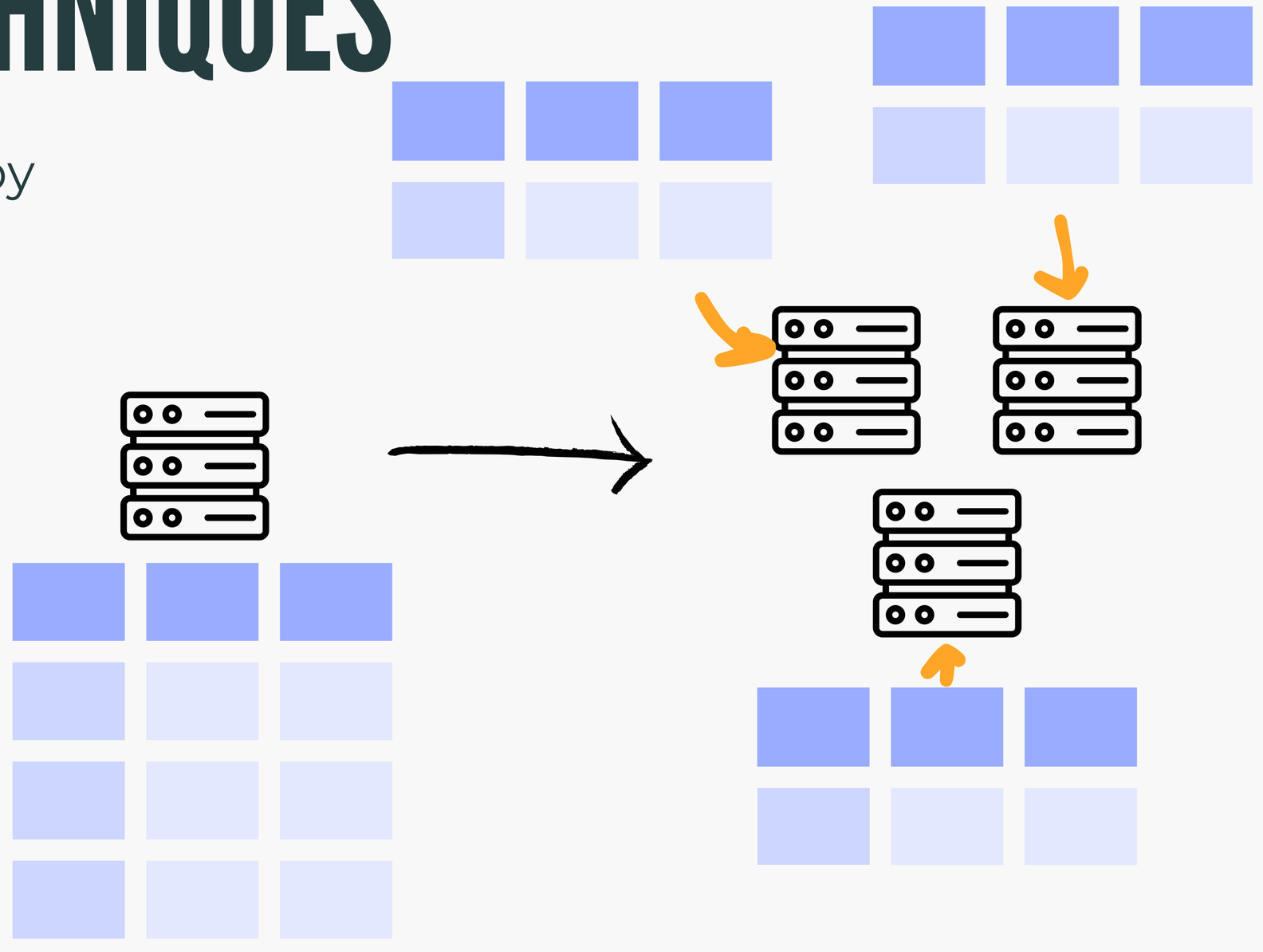
- Tables in relational DBs need to be broken up into **partitions**

- Each partition is then placed into separate serves **(sharding)**



SHARDING TECHNIQUES

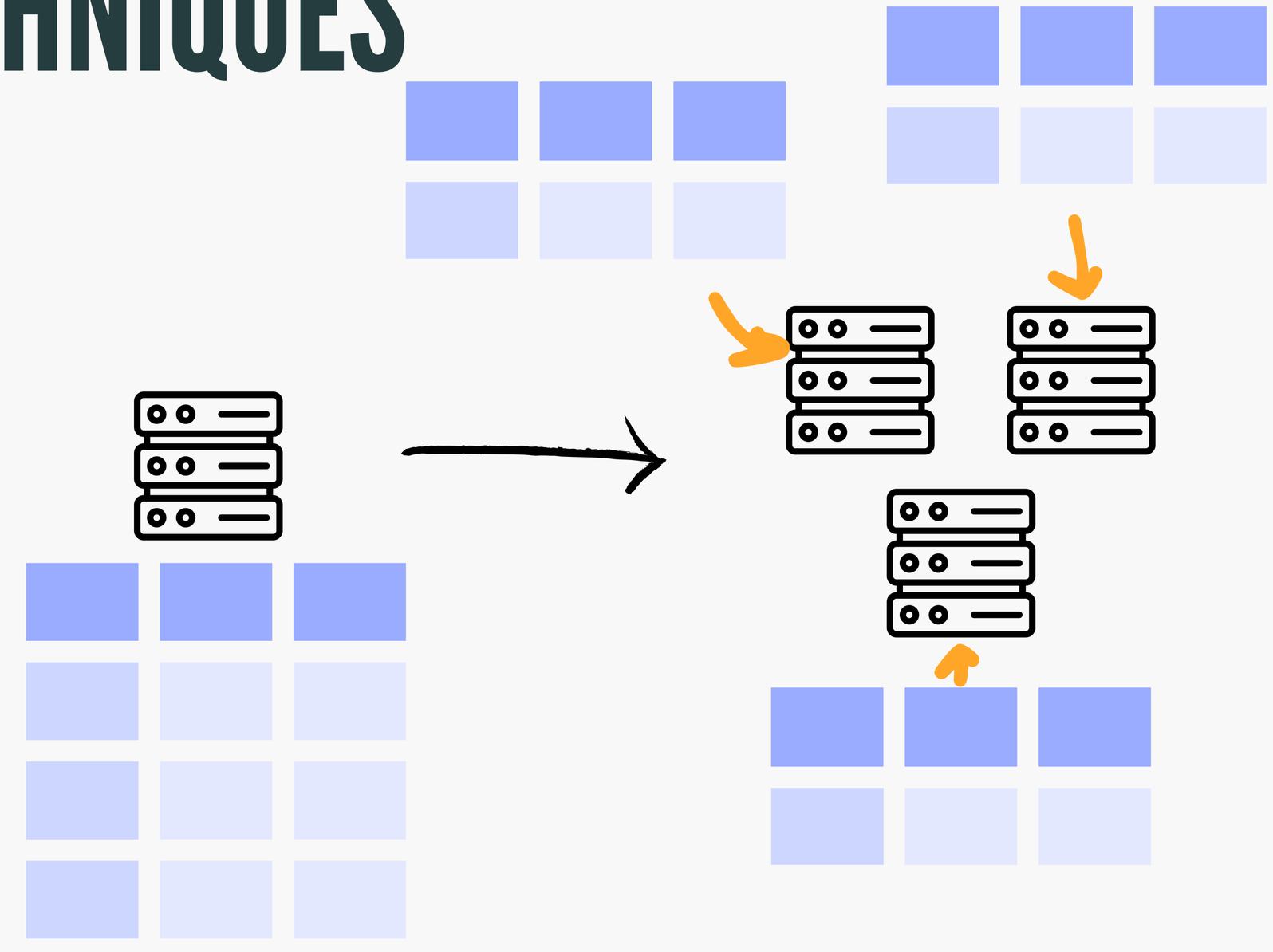
- **Geo-based** - partitioned by user location (east US, or west US)
- **Range based** - based on ranges of key value
- **Hash based** - hash value based on key value then hash value is used for partition



SHARDING TECHNIQUES

Benefits:

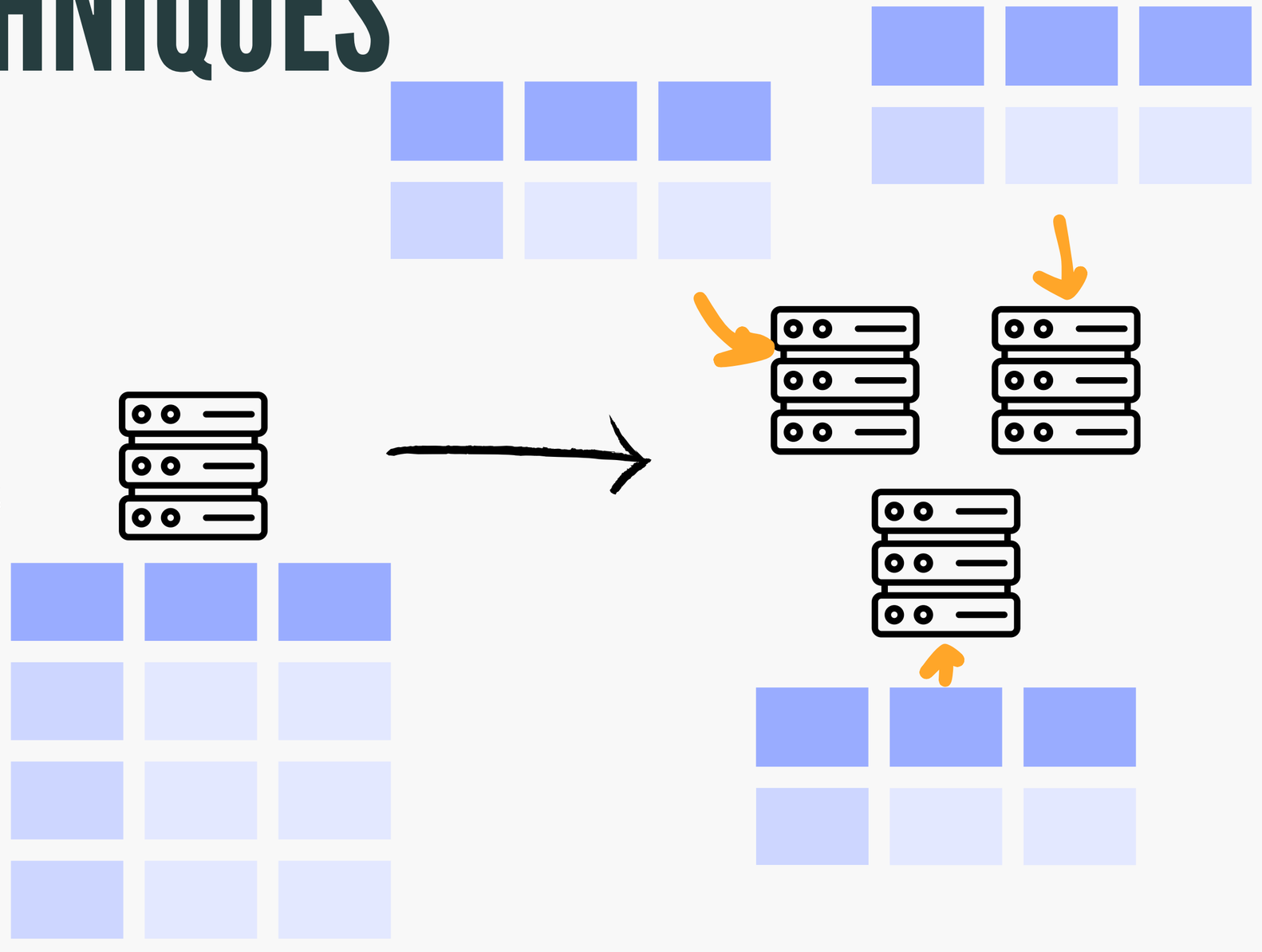
- Faster querying
- System resilience



SHARDING TECHNIQUES

Downsides:

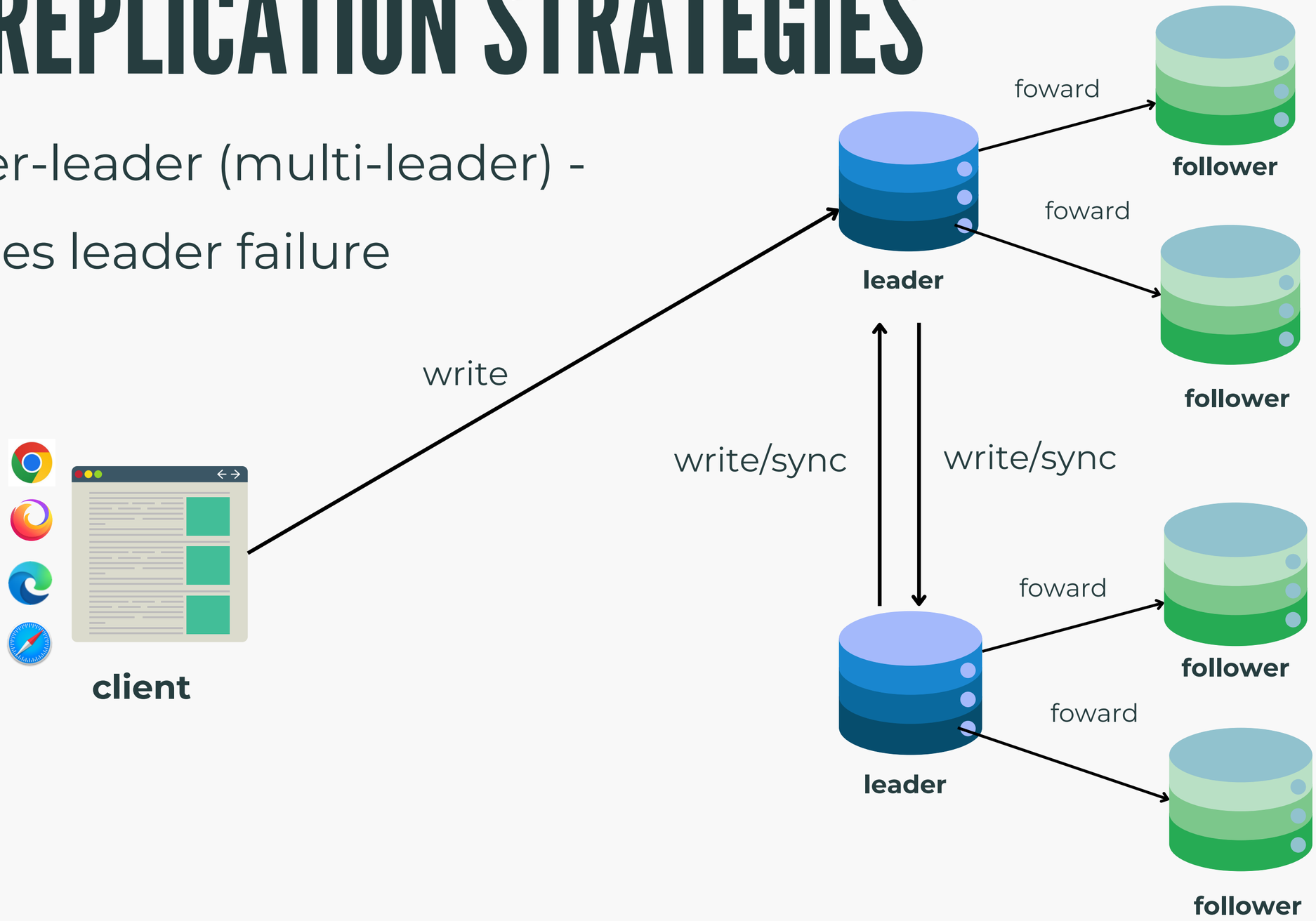
- Difficult to make schema changes
- Foreign key relationships can be complex
- Table joins are expensive
- Need replicas



DB REPLICATION STRATEGIES

Leader-leader (multi-leader) -

handles leader failure



DB REPLICATION STRATEGIES

Leaderless replication - relies on a quorum based approach

write quorum + read quorum > number of nodes for strong consistency

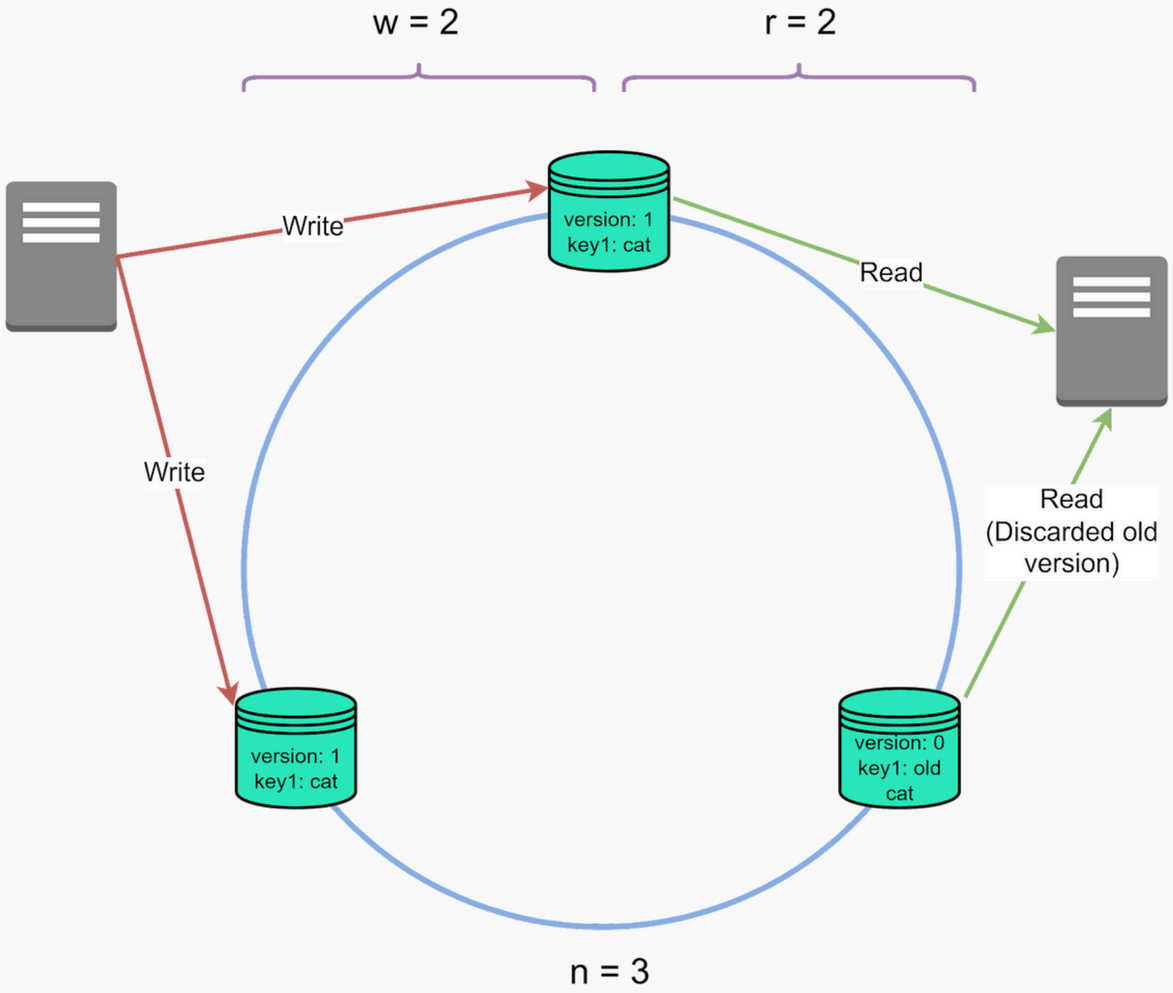


Image source: https://blog.bytebytego.com/p/how-to-choose-a-replication-strategy

SQL VS NOSQL DB

SQL

- All SQL DB are ACID compliant
- Need to know data structure ahead of time
- Difficult to horizontally scale

NOSQL

- Only some NoSQL DBs are ACID compliant
- Good for when format of data is unknown
- Quicker to set up
- Better for data sharding

PUTTING EVERYTHING TOGETHER

Want data at planet scale?

- Eventual consistency is key
- High availability